JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.file

Interface FileVisitor<T>

All Known Implementing Classes:

SimpleFileVisitor

---

public interface **FileVisitor<T>**

A visitor of files. An implementation of this interface is provided to the Files.walkFileTree methods to visit each file in a file tree.

Usage Examples: Suppose we want to delete a file tree. In that case, each directory should be deleted after the entries in the directory are deleted.

```
Path start = ...
Files.walkFileTree(start, new SimpleFileVisitor<Path>() {
    @Override
    public FileVisitResult visitFile(Path file, BasicFileAttributes attrs)
        throws IOException
    {
        Files.delete(file);
        return FileVisitResult.CONTINUE;
    }
    @Override
    public FileVisitResult postVisitDirectory(Path dir, IOException e)
        throws IOException
    {
        if (e == null) {
            Files.delete(dir);
            return FileVisitResult.CONTINUE;
        } else {
            // directory iteration failed
            throw e;
        }
```

```
        }
    });
```

Furthermore, suppose we want to copy a file tree to a target location. In that case, symbolic links should be followed and the target directory should be created before the entries in the directory are copied.

```
    final Path source = ...
    final Path target = ...

    Files.walkFileTree(source, EnumSet.of(FileVisitOption.FOLLOW_LINKS), Integer.M
        new SimpleFileVisitor<Path>() {
            @Override
            public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes
                throws IOException
            {
                Path targetdir = target.resolve(source.relativize(dir));
                try {
                    Files.copy(dir, targetdir);
                } catch (FileAlreadyExistsException e) {
                     if (!Files.isDirectory(targetdir))
                         throw e;
                }
                return CONTINUE;
            }
            @Override
            public FileVisitResult visitFile(Path file, BasicFileAttributes attrs)
                throws IOException
            {
                Files.copy(file, target.resolve(source.relativize(file)));
                return CONTINUE;
            }
        });
```

Since:

1.7

Method Summary

Abstract Methods

Modifier and Type	Method	Description
FileVisitResult	postVisitDirectory(T dir, IOException exc)	Invoked for a directory after entries in the directory, and all of

		their descendants, have been visited.
FileVisitResult	**preVisitDirectory**(**T** dir, **BasicFileAttributes** attrs)	Invoked for a directory before entries in the directory are visited.
FileVisitResult	**visitFile**(**T** file, **BasicFileAttributes** attrs)	Invoked for a file in a directory.
FileVisitResult	**visitFileFailed**(**T** file, **IOException** exc)	Invoked for a file that could not be visited.

Method Detail

preVisitDirectory

```
FileVisitResult preVisitDirectory(T dir,
                                  BasicFileAttributes attrs)
                           throws IOException
```

Invoked for a directory before entries in the directory are visited.

If this method returns `CONTINUE`, then entries in the directory are visited. If this method returns `SKIP_SUBTREE` or `SKIP_SIBLINGS` then entries in the directory (and any descendants) will not be visited.

Parameters:

`dir - a reference to the directory`

`attrs - the directory's basic attributes`

Returns:

`the visit result`

Throws:

`IOException - if an I/O error occurs`

visitFile

```
FileVisitResult visitFile(T file,
                          BasicFileAttributes attrs)
                   throws IOException
```

Invoked for a file in a directory.

Parameters:

`file - a reference to the file`

attrs - the file's basic attributes

Returns:

the visit result

Throws:

IOException - if an I/O error occurs

visitFileFailed

```
FileVisitResult visitFileFailed(T file,
                                IOException exc)
                         throws IOException
```

Invoked for a file that could not be visited. This method is invoked if the file's attributes could not be read, the file is a directory that could not be opened, and other reasons.

Parameters:

file - a reference to the file

exc - the I/O exception that prevented the file from being visited

Returns:

the visit result

Throws:

IOException - if an I/O error occurs

postVisitDirectory

```
FileVisitResult postVisitDirectory(T dir,
                                   IOException exc)
                            throws IOException
```

Invoked for a directory after entries in the directory, and all of their descendants, have been visited. This method is also invoked when iteration of the directory completes prematurely (by a visitFile method returning SKIP_SIBLINGS, or an I/O error when iterating over the directory).

Parameters:

dir - a reference to the directory

exc - null if the iteration of the directory completes without an error; otherwise the I/O exception that caused the iteration of the directory to complete prematurely

Returns:

the visit result

Throws:

```
IOException - if an I/O error occurs
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.